Del texto a una decisión

Usaremos la misma reseña para seguir dos caminos: una LSTM bidireccional con pooling y un Transformer.

FRASE DE EJEMPLO

«La película no me gustó nada»

Objetivo ilustrativo: clasificar el sentimiento de la reseña.

Primero, la frase se convierte en IDs. Para agrupar frases de distintas longitudes, se añade relleno:

La	película	no	me	gustó	nada	PAD	PAD

lengths = 6

pad = (x == PAD)

PAD marca las posiciones de relleno

no es una palabra de la reseña

Dos rutas a partir de los mismos vectores de palabra

RUTA A

LSTM bidireccional + pooling

Embedding → Packed LSTM → Pooling

Avanza palabra por palabra en ambos sentidos. El pooling reúne los vectores en uno.

RUTA B

Transformer encoder

Embedding + posición → Encoder x 2 → Mean pooling

Atiende a las palabras entre sí en cada capa. Luego promedia las posiciones reales.

Tres símbolos para leer las formas

B	L	D / H
cantidad de frases en el lote	largo de la fila, incluido PAD	tamaño de cada vector

Para la LSTM del ejemplo: H=128 en cada dirección, así que cada salida por palabra tiene 2H=256 valores. Para el Transformer: D=d_model=100.

Idea central: ambos modelos reciben IDs y producen una puntuación; cambian la forma de contextualizar las palabras y resumir la frase.

Cómo se prepara y lee la frase

La LSTM necesita saber dónde termina cada reseña antes de recorrerla.

1

Embedding: ID a vector

Cada ID busca una fila de una tabla aprendida. Si hay vectores preentrenados, copy_ los coloca en esa tabla. La fila de PAD se señala con padding_idx.

`nn.Embedding(vocab, 100)`

2

Dropout en los embeddings

Durante el entrenamiento apaga al azar algunos valores de los vectores para reducir la dependencia de señales concretas. En evaluación se desactiva.

`Dropout(0.4)`

3

Empaquetar lo que sí existe

Usa lengths para que la LSTM salte los PAD. lengths.cpu() pasa las longitudes al dispositivo esperado; enforce_sorted=False evita ordenar las frases a mano.

`pack_padded_sequence`

4

Leer en dos sentidos

La dirección directa lee «La ... nada»; la inversa lee «nada ... La». Cada palabra obtiene contexto de ambos lados. Cada dirección tiene H=128 valores.

`bidirectional=True`

5

Volver a la matriz común

Restaura una salida por posición y usa total_length para recuperar el largo L del lote. out queda con forma [B, L, 256].

`pad_packed_sequence`

¿Qué queda en h_n?

En una LSTM bidireccional de una capa, h_n contiene el estado final de cada dirección: forma [2, B, H]. Por eso el código toma h_n[-2] (directa) y h_n[-1] (inversa) cuando elige pooling="last".

Detalle útil: padding_idx evita actualizar la fila PAD por gradiente, pero copiar pesos preentrenados puede cambiar sus valores. Comprueba esa fila y que PAD == pad_idx en las máscaras.

Cuatro maneras de resumir la reseña

Los cuatro caminos reciben los vectores contextuales de la LSTM. Cada camino produce un solo vector por frase.

MEAN

Promedio

Todas las palabras reales aportan por igual.

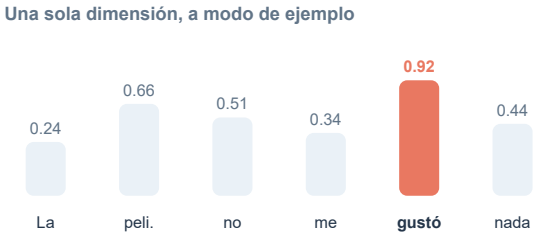


Promedia los seis vectores reales. Es simple y reparte el peso por igual.

MAX

Max pooling

Conserva señales grandes por característica.



El máximo se calcula por cada dimensión; distintas dimensiones pueden venir de palabras distintas. PAD recibe -1e9 antes del máximo para que no gane.

ATTN

Attention pooling

Aprende cuánto pesar cada palabra.



self.attn puntúa cada palabra. La máscara bloquea PAD; softmax(dim=1) hace pesos que suman 1 por reseña. La suma ponderada combina los vectores. Softmax no elige el máximo; last_attn guarda los pesos con detach para inspección.

LAST

Estados finales

Resume desde el final de cada dirección.



torch.cat concatena los dos estados por la dimensión de características. No los suma: junta 128 + 128 = 256 valores.

h = [estado directo | estado inverso]

TODAS LAS RUTAS TERMINAN AQUÍ



fc devuelve un logit por reseña; aquí no se aplica sigmoid para convertirlo en probabilidad.

Embedding + posición: darle orden al texto

La atención por sí sola no sabe qué palabra está en la posición 1 o en la 5. El modelo añade una representación de posición.

MISMA FRASE, AHORA CON ÍNDICES DE POSICIÓN

La	película	no	me	gustó	nada
0	1	2	3	4	5

Las posiciones empiezan en 0. El vector de posición es aprendido y tiene D=100 valores.

1. Embedding de palabra

nn.Embedding(vocab, 100) busca un vector de 100 valores por cada ID. Una matriz preentrenada opcional se copia con copy_.

2. Embedding de posición

pos = nn.Embedding(max_len, 100) busca un vector para cada posición. Se inicializa con valores aleatorios pequeños (std=0.02) y se aprende.

E [B,L,100] + P [L,100] = h [B,L,100]

La suma es elemento a elemento; P se comparte en el lote. Después se aplica dropout.

La máscara de padding

La	película	no	me	gustó	nada	PAD	PAD
						True	True

pad = (x == PAD)

src_key_padding_mask=pad le dice a la atención que no use las posiciones PAD como claves/valores al mirar otras palabras. La máscara tiene forma [B,L].

A diferencia de pack, el Transformer conserva las posiciones y las enmascara. Al final, otra máscara excluye PAD del promedio.

Dos detalles del código

- max_len=400 reserva hasta 400 posiciones; una frase más larga supera la tabla posicional.
- El argumento lengths aparece en forward, pero aquí no se consulta: la máscara se calcula con PAD.

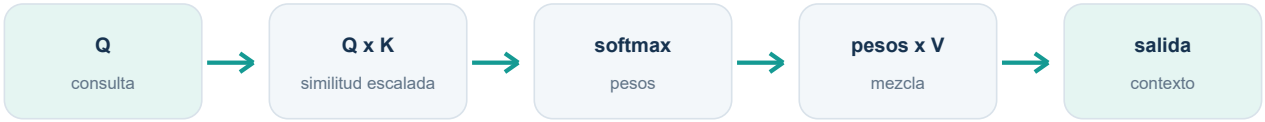
Cómo se miran las palabras entre sí

El encoder repite dos bloques: atención entre palabras y una red pequeña aplicada a cada posición.

BLOQUE 1

Atención propia multi-cabeza

Cada posición crea tres versiones de su vector: Q (qué busco), K (qué ofrezco para comparar) y V (qué información aporto).



Q de una palabra se compara con las K de todas las palabras no enmascaradas. Las puntuaciones se escalan y softmax las normaliza por fila; esos pesos combinan las V.

4 cabezas en paralelo: D=100 se reparte en 4 subespacios de 25 valores cada uno.

Después de la atención

Con norm_first=True el flujo es pre-norm: normaliza la entrada, calcula atención y suma la entrada original (conexión residual).



La salida de este subbloque sigue teniendo forma [B,L,100].

LayerNorm -> Linear -> GELU -> Linear -> suma

Red feed-forward

Se aplica por separado a cada posición. No mezcla palabras entre sí; esa mezcla ya la hizo la atención.



El código usa ff=256 y activación GELU. Una conexión residual y normalización acompañan este subbloque.

capas=2 repite el bloque; LayerNorm final normaliza D=100. Dropout=0.3 regulariza durante el entrenamiento. No hay máscara causal: cada palabra mira ambos lados. enable_nested_tensor=False es una opción interna de ejecución.

Cómo termina el Transformer

Después de contextualizar cada posición, este modelo promedia las palabras reales y calcula una puntuación.

SALIDA DEL ENCODER

h tiene forma [B,L,100]: un vector contextual por posición.

La	película	no	me	gustó	nada	PAD	PAD

vectores reales — relleno

Mean pooling solo sobre palabras reales

m = (~pad).unsqueeze(-1) crea una máscara [B,L,1]. Multiplicar h * m borra PAD; sumar en L y dividir por el número de palabras reales da un vector [B,100].

suma de vectores reales ÷ cantidad de palabras reales

h pooled	→	Dropout	→	fc	→	squeeze
100 valores		durante train		100 a 1		[B]

Salida sin sigmoid: un logit/puntuación por reseña, no una probabilidad.

LSTM y Transformer, lado a lado

Aspecto	LSTM + pooling	Transformer
Contexto	Recorre en dos sentidos	Atiende a todas las posiciones
Relleno	pack usa lengths	máscara bloquea PAD
Orden	lo da la secuencia LSTM	se añade embedding posicional
Resumen	mean / max / attn / last	mean de palabras reales

Cuatro cosas que conviene comprobar

- PAD == pad_idx: el ID de relleno debe coincidir en embedding y máscaras.
- max_len debe ser al menos el largo máximo de entrada (aquí, 400).
- Una matriz preentrenada debe respetar el orden del vocabulario y la dimensión configurada.